

Learning to Act While Waiting: RL Finetuning of Generalist Robot Policies Under Inference Latency

Brian Zhu^{*1} Momen Khalil^{*1} E Harrison^{*2} Emanuele Poggi^{*1} Philipp Schmitt¹ Bernd Kast¹ Philine Meister¹ Pranav Atreya² Qiyang Li² Finn Ferchau¹ Cesar Colmenero¹ Yash Shahapurkar¹ Gokul Narayanan¹ Melih Erdogan¹ Kai Wurm¹ Georg von Wichert¹ Oier Mees^{3,4,2} Eugen Solowjow¹ Andrew Wagenmaker² Sergey Levine²

¹ Siemens

² UC Berkeley

³ Microsoft

⁴ ETH Zurich

Abstract: While reinforcement learning (RL) allows generalist robot policies to continually improve during deployment, the large model size of modern generalist policies, such as VLAs, poses a fundamental obstacle to effective RL improvement. In particular, their severe inference latency—which can lead to pauses or jerky movements—can alter the effective environment dynamics and, if not correctly accounted for, break the Markov assumption that RL relies on, causing standard RL algorithms to fail completely. In this work, we introduce a latency-aware framework, *Asynchronous RL with Intermediate Information* (ARLI), that enables RL-based improvement of generalist policies under inference delays. Our framework builds on asynchronous inference approaches, which interleave action generation with execution to hide latency, and addresses its incompatibility with RL by providing a low-latency RL policy design that maximizes reactivity within the inference window through two contributions: state augmentations that restore near-Markovian structure by incorporating committed actions and a mid-inference observation. We evaluate our approach across simulated and real-world manipulation tasks, and find that it enables effective finetuning under inference delays where standard RL fails entirely, even matching or exceeding the performance of standard RL in idealized no-latency settings.

Keywords: RL Steering, Inference Latencies, Real-time Control

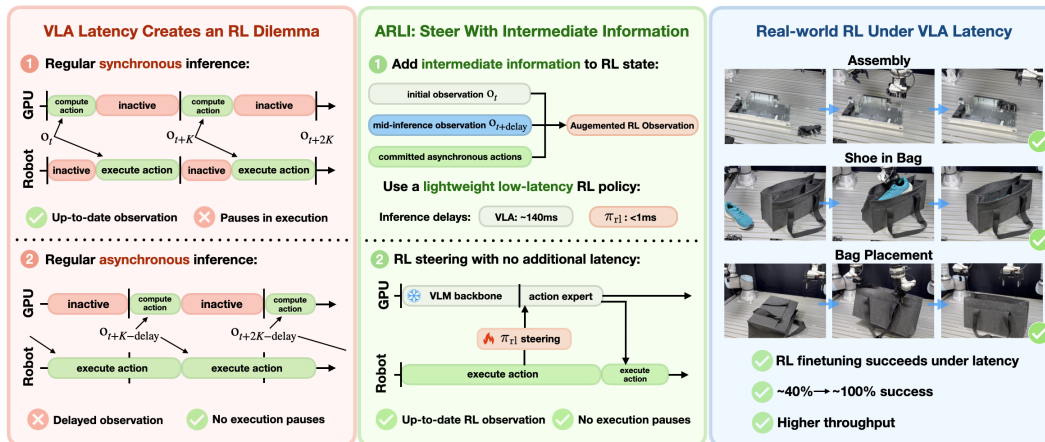


Figure 1: **Enabling RL improvement of generalist robot policies under inference latency:** Generalist policies incur significant action inference latency delays, either forcing pauses during inference or relying on stale observations, making RL improvement brittle. ARLI uses intermediate actions and updated asynchronous observations to steer the next action chunk, enabling effective real-world improvement under asynchronous inference, and real-world VLA improvement.

*Equal Contribution

1 Introduction

Enabling robots to learn and improve through experience collected in deployment has proven a critical step in achieving highly performant robotic behaviors. In particular, with the advent of *generalist* robot policies—policies trained on diverse, large-scale datasets to perform a variety of tasks [1–11]—online improvement and adaptation approaches have enabled significantly more reliable and high-precision behaviors than are typically achievable via offline training alone [12–17]. While significant progress has been made developing RL-based approaches for improving generalist policies, the most promising results remain limited to constrained, lab-like settings. As generalist policies are increasingly deployed in real-world settings, enabling RL improvement under practical deployment constraints is critical to achieving effective real-world robotic control.

A key constraint present in the deployment of generalist policies is *inference latency*. Generalist policies, such as vision-language-action models (VLAs), are typically very large—commonly billions of parameters [5–7, 11]—and generating an action with a generalist policy can take a non-trivial amount of time. For example, on a standard consumer-grade GPU, running a single action generation step on π_0 , a commonly used open-source VLA, takes approximately 100 milliseconds [6], while other open-source models can take over 300 milliseconds per action generation [5, 18]. Such inference delays can lead to pauses or jerky movements in deployment, altering the effective dynamics of the environment and leading to poor policy performance, especially in settings that require high reactivity. While a growing body of work has sought to address challenges introduced by policy latency [19–21], these works typically focus simply on policy *deployment*, and do not investigate how latency affects the ability of RL-based approaches to learn effectively.

In this work we seek to close this gap and enable effective RL improvement of generalist policies under inference latency constraints. As a starting point, we build on several recent works that propose *asynchronous inference* strategies, where the generalist policy begins computing the next action while the current action is still being executed [20, 21]. While asynchronous inference, if correctly instantiated, has been shown to significantly mitigate the effects of inference latency, it comes at a cost: if we begin policy inference while the previous action is being executed, we cannot condition on the actual environment state we will be at when we play this action, since we have not yet reached this state. As a result, naively combining RL improvement with asynchronous inference leads to a *non-Markovian effective state*, and causes standard RL approaches to fail to learn.

To address this challenge and enable RL finetuning with asynchronous inference, we develop ARLI, or *Asynchronous RL with Intermediate Information*, that overcomes the non-Markovian state with careful state augmentation. In particular, ARLI is inspired by two key insights. First, by including the *intermediate actions* in the state, we can give the RL policy significantly more predictive information about future states, allowing it to more accurately finetune the generalist policy’s actions. Second, in many cases, *RL inference is required at a later stage than generalist policy inference*, and we can thus integrate the RL corrections at a later time than when we begin generalist policy inference. Given this, the RL policy can be conditioned on a more up-to-date *intermediate state* than what the generalist policy has access to, mitigating the effect of the asynchronous inference delay.

We show that ARLI leads to effective improvement of generalist policies under practical latency constraints. In particular, we evaluate ARLI on several simulated tasks where high reactivity is required, and find that it yields significantly more efficient RL improvement, in many cases enabling improvement when naive approaches to combining RL with asynchronous inference fail. We then show that this holds in three challenging real-world tasks on a bimanual UR5e robot, finding that ARLI enables real-world policy improvement under asynchronous inference in regimes where both naively combining RL with asynchronous inference, and attempting to use synchronous inference, fail.

2 Related Work

Robot policy deployment under latency constraints. With the advent of “generalist” robotic foundation models, inference delays have become a central concern in policy deployment, as the forward passes of such policies routinely exceed the controller’s sampling period [5, 6, 22–25].

Action chunking [26] amortizes inference cost by producing k actions per forward pass, and is now standard in modern robot policies [3, 6, 10, 27–29]. Action chunking only partially addresses the latency problem, however, as synchronous execution incurs periodic pauses between chunks, yet naive asynchronous execution introduces discontinuities at chunk boundaries [19, 21]. Recent work mitigates these discontinuities through temporal ensembling [26], rejection sampling [30], or diffusion inpainting [21]. A separate line of sim-to-real work injects delays into simulation during training to obtain delay-robust policies [31, 32]. All of these methods, however, target deployment of a fixed pretrained policy and do not consider online adaptation.

RL under latency constraints. Prior work has studied reinforcement learning and control under delayed information, actions, and rewards, from early formulations of delayed closed-loop control and delayed-state/action MDPs [33–36] to more recent methods for stochastic delays and real-time decision making [37, 38]. Closely related work also considers concurrent control, where agents continue acting while new decisions are being computed [39]. These works largely consider generic RL settings or train policies from scratch, and do not address the action-chunked asynchronous regime of modern diffusion and VLA policies, the setting we target in this work.

RL for robotics. Reinforcement learning has played a key role enabling highly performant policies for robotic control, spanning domains from locomotion [40, 41] to manipulation [42–46]. A common strategy to improve the sample efficiency of RL is to rely on simulators, transferring sim-learned policies to the real world [47–53], yet such approaches are often limited by the sim-to-real gap [54]. With the advent of generalist robot policies, significant focus has been devoted to enabling improvement and adaptation of such policies [12–15, 55–68]. In this work we build on [12], which enables RL improvement of pretrained policies by steering the input noise to the policy’s denoising process (in the case when the pretrained policy is a diffusion or flow model). However, none of these works explicitly deal with enabling RL finetuning under latency constraints.

3 Preliminaries

We consider decision-making in Markov decision processes (MDPs). An MDP is denoted by a tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, P_{\text{init}}, r, \gamma)$ where $\mathcal{S}$ is the state space, $\mathcal{A}$ the action space, $P : \mathcal{S} \times \mathcal{A} \rightarrow \Delta_{\mathcal{S}}$ the transition kernel, $P_{\text{init}} \in \Delta_{\mathcal{S}}$ the initial state distribution, $r : \mathcal{S} \rightarrow \mathbb{R}$ the reward, and $\gamma \in [0, 1]$ the discount factor. Interaction with the MDP proceeds in episodes. First the environment samples an initial state $s_1 \sim P_{\text{init}}$, the agent then selects some action $a_1 \in \mathcal{A}$, the environment transitions to state $s_2 \sim P(s_1, a_1)$, and so forth, proceeding until the episode terminates. A policy denotes a mapping from states to actions, $\pi : \mathcal{S} \rightarrow \Delta_{\mathcal{A}}$. For a policy π and state $s \in \mathcal{S}$, the value function quantifies the expected discounted reward, $V^\pi(s) := \mathbb{E}^\pi[\sum_{t=1}^{\infty} \gamma^{t-1} r(s_t) \mid s_1 = s]$, where $\mathbb{E}^\pi[\cdot]$ denotes the expectation over trajectories induced by executing π on $\mathcal{M}$. The value of a policy, $V(\pi) := \mathbb{E}_{s_1 \sim P_{\text{init}}}[V^\pi(s_1)]$, denotes its expected reward over trajectories. The typical goal of RL, and the goal we consider in this work, is to learn a policy that maximizes $V(\pi)$.

Pretrained policies and inference delays. In this work, we assume we are given an initial pretrained policy π_{pt} , for example, a policy trained via behavioral cloning on human demonstrations. We make several assumptions on π_{pt} . First, we assume that instead of predicting a single action at each step, π_{pt} predicts an *action chunk*, a sequence of k actions, $A = (a_1, \dots, a_k)$. Typically, after producing A the policy executes the first n steps in the chunk before recomputing a new chunk. Second, we assume that π_{pt} is a diffusion or flow policy [27]. As such, π_{pt} not only takes a state s as input, but a random noise w , typically sampled from $\mathcal{N}(0, I)$. We note that both of these assumptions are standard design choices in modern robot learning, so this is not a major restriction [3, 6–11]. When π_{pt} is a VLA or other large model, simply computing $A_t = \pi_{\text{pt}}(s_t, w_t)$ for $w_t \sim \mathcal{N}(0, I)$ can take a significant amount of time. Formally, we assume that we can generate an action from π_{pt} but that this generation will require t_{delay} steps (for simplicity, we assume $t_{\text{delay}} < k \cdot \Delta_a$, where Δ_a is the time interval we execute each action). We will overload notation somewhat and let t_{delay} refer to both the inference delay (in seconds) as well as the number of environment “steps” it occupies (where each environment step is an interval of length Δ_a).

asynchronous inference, as outlined in Section 3, is required to enable effective performance. We therefore focus on the asynchronous inference setting, and how we can enable efficient online improvement under asynchronous inference. In the following, we outline our proposed approach, asynchronous RL with intermediate information (ARLI); see Figure 2 for an overview.

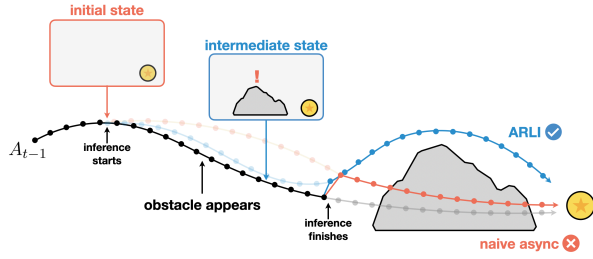
4.1 Enabling DSRL Finetuning with Inference Latency

To enable efficient finetuning of π_{pt} , we consider DSRL as our core RL algorithm. Assume that we begin the inference of π_{pt} at step $t - t_{\text{delay}}$ and condition it on state $s_{t-t_{\text{delay}}}$. Naive application of DSRL to finetune π_{pt} would condition the noise policy π_{rl} on the state that π_{pt} is conditioned on—in this case $s_{t-t_{\text{delay}}}$ —and generate an initial noise to steer the action generation of π_{pt} based on this state. In the asynchronous inference setting, however, the computed action is not played until step t —there is a delay of t_{delay} steps between when the RL policy computes an action and this action is played—and, as we will see in Section 5, naive application of DSRL in the asynchronous inference setting therefore does not lead to effective finetuning. To enable effective RL improvement under asynchronous inference, we consider three key modifications, outlined below.

ARLI actions: Conditioning π_{rl} on intermediate actions. Even though the most current state we have access to when inference begins is $s_{t-t_{\text{delay}}}$, we also have access to the *intermediate actions*, $a_{t-t_{\text{delay}}:t} := (a_{t-t_{\text{delay}}}, a_{t-t_{\text{delay}}+1}, \dots, a_{t-1})$ —the actions from the previously computed action chunk that will be executed during inference. While not fully predictive of the state at time t in general, knowing $s_{t-t_{\text{delay}}}$ and $a_{t-t_{\text{delay}}}, a_{t-t_{\text{delay}}+1}, \dots, a_{t-1}$ gives significant information about where we will be at time t , as we can then estimate how the environment evolves from $s_{t-t_{\text{delay}}}$.

Our first modification is to condition π_{rl} not just on $s_{t-t_{\text{delay}}}$, but also the intermediate actions $a_{t-t_{\text{delay}}:t}$ (see part (a) of Figure 2). That is, denoting the “RL state”—the state we condition the RL policy on—as s_t^{rl} , we set $s_t^{\text{rl}} \leftarrow (s_{t-t_{\text{delay}}}, a_{t-t_{\text{delay}}:t})$. Note that while it is not straightforward to condition π_{pt} on intermediate actions (since π_{pt} is a frozen policy that cannot easily incorporate additional conditioning information), π_{rl} can learn to utilize such information as needed, adapting its behavior as it learns how $a_{t-t_{\text{delay}}:t}$ affects the future state.

ARLI state: Conditioning π_{rl} on intermediate state by decomposing inference delays. While incorporating the intermediate actions into the RL state enables more effective action selection to be played at a future state, it does not account for potential disturbances that may occur between $t - t_{\text{delay}}$ and t ; see Figure 3 for an example of this scenario.



To allow for reactivity to such perturbations, we note that most VLAs are decomposed into two components: the *VLM backbone* ($\pi_{\text{pt}}^{\text{vlm}}$) which takes as input images and language and outputs a sequence of embeddings, and the *action expert* ($\pi_{\text{pt}}^{\text{ae}}$) which instantiates the denoising process, taking as input the embeddings produced by $\pi_{\text{pt}}^{\text{vlm}}$ and producing the final action. In many cases, $\pi_{\text{pt}}^{\text{vlm}}$ is significantly larger and takes longer to run inference for than $\pi_{\text{pt}}^{\text{ae}}$; the VLA π_0 spends approximately 2/3 of each inference call on inference of $\pi_{\text{pt}}^{\text{vlm}}$, and only the last 1/3 on $\pi_{\text{pt}}^{\text{ae}}$ inference [6].

Since DSRL modifies the behavior of π_{pt} by selecting the input noise for the denoising process (which is only initialized once $\pi_{\text{pt}}^{\text{ae}}$ is called), we can begin inference on π_{rl} right before $\pi_{\text{pt}}^{\text{ae}}$ is called to gain access to a more recent state. If we denote the combined inference time of π_{rl} and $\pi_{\text{pt}}^{\text{ae}}$ as $t_{\text{delay}}^{\text{rl}}$, the latest we can call π_{rl} is at timestep $t - t_{\text{delay}}^{\text{rl}}$ without incurring additional inference delay. Therefore, we propose further conditioning π_{rl} on the state at this timestep (see part (b) of Figure 2),

transforming our RL state to be $s_t^{\text{rl}} \leftarrow (s_{t-t_{\text{delay}}}, a_{t-t_{\text{delay}}}, s_{t-t_{\text{delay}}}^{\text{rl}})$. Although this intermediate state is still slightly delayed, the dominant latency comes from the VLM backbone, making s_t^{rl} substantially more predictive of s_t compared to using only the initial state and intermediate actions.

RL with RTC. Although asynchronous inference does not require RTC, prior work has shown that RTC improves zero-shot policy deployment by encouraging temporal consistency between predicted action chunks and reducing discontinuities across inference calls. While such behavior could in principle be learned by π_{rl} , explicitly enforcing it through RTC ensures this behavior is reliably executed without affecting the learning speed. Specifically, RTC freezes the noise for the first t_{delay} denoising steps and applies an inpainting guidance term to maintain continuity in later steps. In our approach, we select the initial noise using DSRL while still incorporating RTC’s inpainting guidance during denoising. We emphasize that, while RTC incorporates intermediate actions in the VLA’s inference, it is *only* to preserve continuity between action chunks; in contrast, π_{rl} can leverage them to learn substantially richer behaviors.

4.2 RL Under Inference Delays Provably Succeeds

While ARLI seeks to condition the RL policy on as up-to-date state information as possible, we must still plan based on a state delayed by t_{delay} steps. We next show that, under certain environment conditions, this will not result in significant policy suboptimality:

Definition 1 (Delayed Oracle Optimality Gap). *An MDP $\mathcal{M}$ exhibits ω_d -delayed optimality gap if for any $s_t, a_{t:t+k}$,*

$$\left| \underbrace{Q_{\text{ac}}^*(s_t, a_{t:t+k})}_{(a)} - \underbrace{\mathbb{E}_{P(\cdot|s_t, a_{t:t+k})}[R_{t:t+k-d} + \gamma^{k-d} V_{\text{ac}}^*(s_{t+k-d}, a_{t+k-d:t+k})]}_{(b)} \right| \leq \omega_d, \quad (1)$$

where Q_{ac}^* denotes the optimal Q -function for action chunk k policies, and we define $V_{\text{ac}}^*(s_t, a_{t:t+d}) = \max_{a_{t+d:t+k}} Q_{\text{ac}}^*(s_t, a_{t:t+k})$.

This quantifies the difference in performance between the optimal achievable performance playing k -step action chunks A_t given the current observation s_t (term (a)), and the performance if we fully commit to the same action chunk as (a), but then make a decision for the next action chunk d -steps early, based on the delayed observation at that step (term (b)). The following result shows that we can apply standard Q -learning approaches to learn effective policies despite our t_{delay} observation delay.

Proposition 1 (Informal). *Consider applying standard Q -learning to observations delayed by t_{delay} steps and action chunks of length k . Then the policy learned through this, $\hat{\pi}^{t_{\text{delay}}}$, satisfies: $V_{\text{ac}}^* - V(\hat{\pi}^{t_{\text{delay}}}) \leq \frac{1}{1-\gamma^{k-t_{\text{delay}}}} \cdot \omega_{t_{\text{delay}}}$ for V_{ac}^* the optimal value of a policy that places action chunks of length k with no delay, and $\omega_{t_{\text{delay}}}$ the oracle optimality gap of $\mathcal{M}$ under delay $d \leftarrow t_{\text{delay}}$.*

Proposition 1 shows that, as long as observation delays do not significantly impact the performance of the *oracle* action chunking policy, we can apply standard Q -learning approaches (in particular, the action chunking Q -learning approach of [72]) to learn a policy that will converge to approximately the loss in performance the oracle policy incurs. As ARLI adopts a Q -learning approach analogous to [72], this shows that, while ARLI may incur some suboptimality from delayed observations, this suboptimality can be effectively bounded. Furthermore, by mitigating the delay with intermediate states, ARLI can reduce the delayed oracle optimality gap, leading to more effective performance than can be achieved without this. We present the full version of Proposition 1 in Section D.

5 Experimental Results

We next test whether ARLI enables effective improvement of generalist policies in practice. We aim to evaluate whether standard RL approaches enable effective improvement under asynchronous VLA inference and whether ARLI is able to improve on these approaches, leading to both faster learning and higher final success rate. We evaluate ARLI on several simulated tasks that require high reactivity, as well as on three real-world tasks.

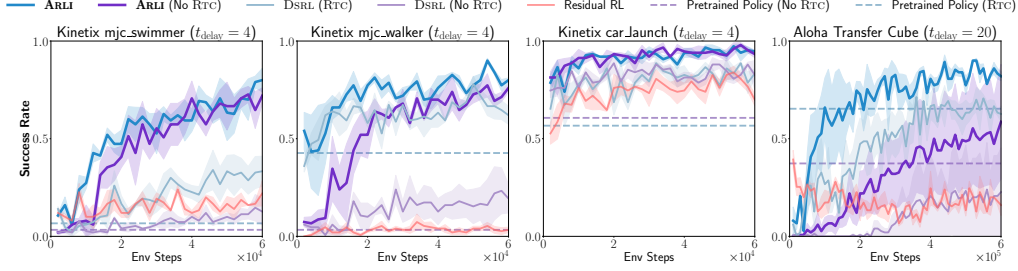


Figure 4: Comparison of ARLI and DSRL on simulation tasks with and without RTC.

Baseline methods. We compare ARLI to several other approaches that integrate RL into asynchronous VLA inference. First, we consider the most straightforward application of asynchronous DSRL, where we condition the RL policy on $s_{t-t_{\text{delay}}}$. We then consider variants of DSRL with all possible state augmentations introduced in Section 4, including intermediate actions, intermediate states, RTC, and different combinations thereof. For our main simulation results, we consider the direct application of DSRL with and without RTC and ARLI with and without RTC; we leave other comparisons for the ablations (Section 5.3). For real-world experiments, we compare against a synchronous inference approach which simply pauses to compute each new action. We run ARLI as described in Section 4.

In addition to variants of DSRL, we also compare ARLI to residual RL [60, 69–71]. Residual RL operates by training an RL policy π_{rl} to output a *residual correction* a_t^{Δ} , and then executing the action $a_t + a_t^{\Delta}$, for a_t the action produced by the base policy. Unlike DSRL which must be incorporated into the inference process of the base policy, residual RL can be run independently of this inference. As π_{rl} is often parameterized as a small MLP that has minimal inference cost, we can compute a fresh residual correction at each step based on the current state, effectively mitigating the effect of the inference delay. For our experiments, we utilize the instantiation of residual RL proposed in [71], which conditions π_{rl} on s_t and a_t , and computes a residual correction at each step.

For all simulation results, we average results across 3 seeds and report error bars denoting 1 standard error. Please see the Appendix for additional experimental details.

5.1 Simulated Experiments

We first evaluate ARLI on several simulated tasks. We consider the **Kinetix** benchmark [21], which contains dynamic environments, requiring high reactivity. For our pretrained policy π_{pt} we utilize the publicly available flow policy checkpoints considered in [21]. For our comparison, we select the `mjc_swimmer`, `mjc_walker`, and `car_launch` tasks, which have lower pretrained policy success rates, and show significant degradation in zero-shot performance over increasing inference latencies. For each task, we set $t_{\text{delay}} = 4$ and $t_{\text{delay}}^{\text{rl}} = 1$. We also consider the **AlohaTransferCube** task [73], which simulates VLA inference with larger latencies and tests ARLI’s ability to scale to more challenging bimanual manipulation tasks. For π_{pt} we utilize the publicly available Aloha finetune of π_0 [6], a 3.3B-parameter VLA. We set $t_{\text{delay}} = 20$ and $t_{\text{delay}}^{\text{rl}} = 10$.

Figure 4 plots policy success rate over time for the 4 simulated tasks, and shows that ARLI significantly outperforms naive application of DSRL under asynchronous inference, converging to both a higher final success rate and requiring less training time. While running DSRL with RTC does improve the finetuning performance on some tasks, we find that this is insufficient to fully learn the desired behavior, and that the state augmentations incorporated in ARLI are required. Furthermore, we see that, while in many cases ARLI is able to perform effectively without RTC, RTC does improve the efficiency and reliability of ARLI. We see as well that ARLI enables significant improvements over residual RL—despite the higher reactivity residual RL enables, incorporating DSRL’s ability to steer the denoising process enables significantly higher final success once we incorporate ARLI’s state augmentations. These results illustrate that ARLI is able to effectively handle the asynchronous

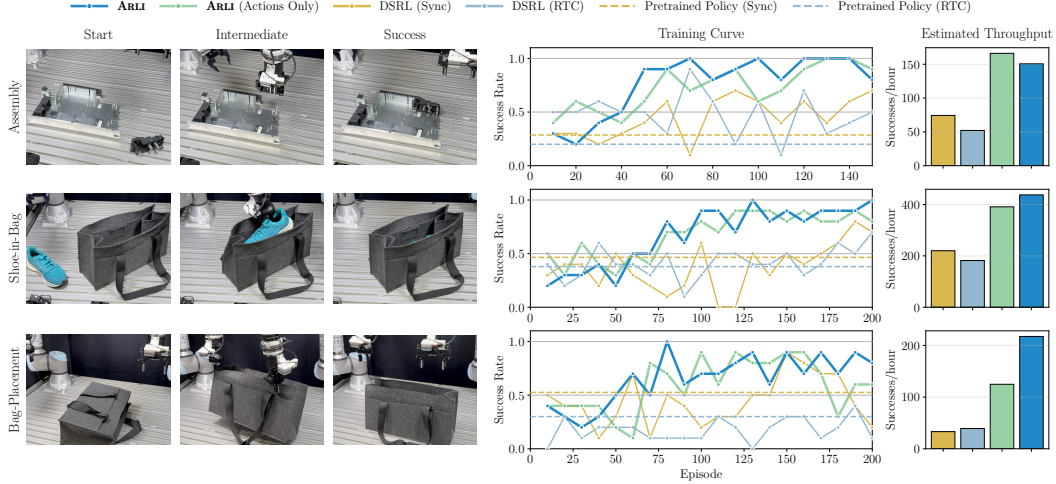


Figure 5: Comparison of ARLI against other RL finetuning methods on real-world tasks. *Left*: task images. *Middle*: training success curves corresponding to average success rate in the last 10 episodes. *Right*: estimated throughput in successes/hour computed based on the duration of each success in the last 20 episodes and assuming failure is declared after 2x average success duration.

inference delays, and still enable highly effective finetuning, both in high reactivity settings, as well as with large-scale VLAs.

5.2 Real-World Experiments

We next test whether ARLI allows for effective RL finetuning in real-world robotic deployment. We consider the following real-world tasks (see Figure 5 for visualization):

- *Assembly*. Insert a power connector into an inverter box by aligning it with two pins.
- *Shoe-in-Bag*. Pick up a shoe and place it into a narrow bag.
- *Bag-Placement*. With both arms, pick up a lying bag and position it parallel to the front.

We run all tasks on a bimanual UR5e robot cell. Each robot is equipped with a wrist camera and a base camera is installed at the front of the cell. For all experiments, we initialize the base policy with $\pi_{0.5}$ [10] finetuned on a small dataset of human demonstrations on the target task at 60hz and action chunk length 50. On an NVIDIA GeForce RTX 5090 GPU, we find $t_{\text{delay}} = 10$, $t_{\text{delay}}^{\text{r1}} = 7$, and action horizon $n = 20$ to be optimal for both sync and RTC policies. Note that with RTC, inference time needed for $\pi_{\text{pt}}^{\text{ae}}$ increases, and we find that $\pi_{\text{pt}}^{\text{vlm}}$ inference takes up only 40% of the total inference time.

Figure 5 shows the results of training on these three tasks. Starting from a base policy success rate of around 40%, ARLI is capable of achieving near 100% success on all tasks after 100 to 125 episodes of training. This is a significant improvement over DSRL (synchronous) and DSRL (RTC), which struggle to reach 80% in the same amount of time on the *Assembly* and *Bag-Placement* tasks, and take nearly double the time to match ARLI in the *Shoe-in-Bag* task. Interestingly, ARLI (Actions Only) performs comparably to ARLI with the exception of the *Bag-Placement* task, where it fails to converge to a high success rate. We highlight that without asynchronous inference, both the base policy exhibits poor success and RL training is significantly slower, or does not learn at all. In other words, asynchronous inference is required to learn on these tasks, and ARLI is capable of facilitating RL finetuning with asynchronous inference, allowing for online VLA improvement in settings where this would previously have not been possible. We also estimate policy throughput in successes/hour and find that ARLI shows significantly higher throughput across all tasks.

5.3 Ablations

We next investigate the efficacy of our design choices through a series of ablation studies.

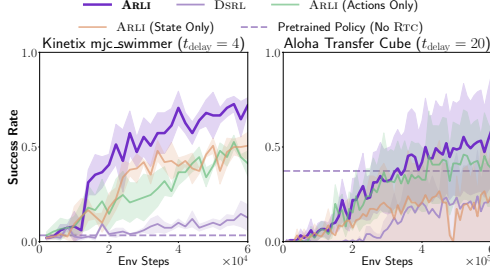


Figure 6: Variants of ARLI (no RTC) with one of intermediate state or actions on `mjc_swimmer` and `AlohaTransferCube`.

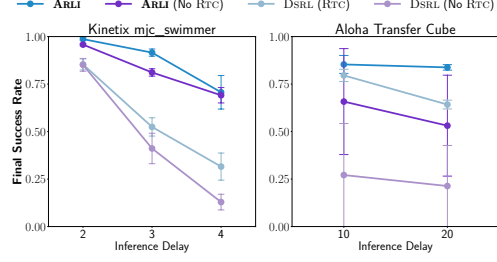


Figure 7: Comparison of ARLI and DSRL of inference delays on `mjc_swimmer` and `AlohaTransferCube`.

How critical is it that we condition π_{rl} on the intermediate state and actions? We evaluate the importance of including intermediate state and actions by running the experiment of Section 5.1 on the `mjc_swimmer` and `AlohaTransferCube` tasks, isolating the usage of the intermediate state and actions. We run these experiments *without* RTC to minimize the temporal information utilized by default. Figure 6 shows that conditioning π_{rl} on both intermediate states and actions yields significantly better results than conditioning on one of the two, highlighting the importance of our choice of s_t^{rl} .

How sensitive is ARLI to t_{delay} ? We evaluate the sensitivity of ARLI to t_{delay} by running the same experiment in Section 5.1 for the `mjc_swimmer` and `AlohaTransferCube` tasks, setting $t_{delay} = [2, 3, 4]$ for Kinetix and $t_{delay} = [10, 20]$ for AlohaTransferCube. We accordingly adjust the length of the intermediate actions to t_{delay} and set $t_{delay}^{rl} = t_{delay} - 1$ for Kinetix and $t_{delay}^{rl} = t_{delay}/2$ for AlohaTransferCube. See Figure 7 for plots of final success rate of ARLI against the baselines. Across both tasks, we see that with RTC the performance of ARLI decays slower than DSRL as t_{delay} increases, indicating that ARLI is more robust to inference delay.

How sensitive is ARLI to t_{delay}^{rl} ? We evaluate the sensitivity of ARLI against RL inference latency by running the same experiment in Section 5.1 for the `mjc_swimmer` task, setting $t_{delay} = 4$ and ablating t_{delay}^{rl} from 0 (most up-to-date) to 4 (no additional information). For these experiments we run ARLI with RTC. See Figure 8 for results, which show that having more up-to-date information is better, and that when t_{delay}^{rl} is greater than half of t_{delay} , learning significantly improves.

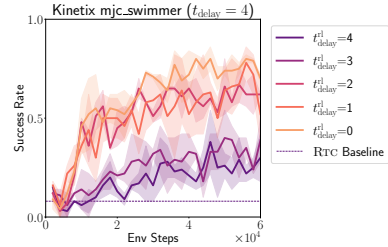


Figure 8: ARLI performance at varying RL inference delays on `mjc_swimmer`.

6 Conclusion and Limitations

In this work, we have proposed ARLI, an approach to enable effective RL improvement of generalist robot policies under practical inference constraints. In particular, we have shown that, by properly augmenting the observations the RL policy is conditioned on, we can enable effective RL improvement with asynchronous policy inference, enabling learning while mitigating the effects of latency.

While ARLI has shown effective performance on our simulated and real-world tasks, it does possess a number of limitations. A key assumption we make for our pretrained policy is that it can be split up into VLM backbone and action expert components, where the action expert is the only part of the policy that requires a noise input. While this is true for many popular VLAs, for policies that do not satisfy this constraint, we cannot utilize intermediate states to improve performance (though ARLI can still take advantage of intermediate actions in this case, as our experimental results demonstrate). While introducing RTC to our method shows performance gains, utilizing RTC might in fact hurt the reactivity of our policy, as it forces old actions to be played, making the RL policy less expressive. Lastly, while DSRL is capable of learning in many tasks, previous works have observed that DSRL can struggle to learn on certain tasks. Given ARLI’s reliance on DSRL, ARLI will likely also struggle to learn on such tasks.

Acknowledgments

This research was partially supported by Siemens and ONR N00014-25-1-2060.

References

- [1] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, J. Hsu, et al. Rt-1: Robotics transformer for real-world control at scale. *arXiv preprint arXiv:2212.06817*, 2022.
- [2] J. Gu, S. Kirmani, P. Wohlhart, Y. Lu, M. G. Arenas, K. Rao, W. Yu, C. Fu, K. Gopalakrishnan, Z. Xu, et al. Rt-trajectory: Robotic task generalization via hindsight trajectory sketches. *arXiv preprint arXiv:2311.01977*, 2023.
- [3] Octo Model Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, C. Xu, J. Luo, T. Kreiman, Y. Tan, P. Sanketi, Q. Vuong, T. Xiao, D. Sadigh, C. Finn, and S. Levine. Octo: An open-source generalist robot policy. In *Proceedings of Robotics: Science and Systems*, Delft, Netherlands, 2024.
- [4] R. Doshi, H. Walke, O. Mees, S. Dasari, and S. Levine. Scaling cross-embodied learning: One policy for manipulation, navigation, locomotion and aviation. In *Conference on Robot Learning*, 2024.
- [5] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.
- [6] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- [7] J. Bjorck, F. Castañeda, N. Cherniadev, X. Da, R. Ding, L. Fan, Y. Fang, D. Fox, F. Hu, S. Huang, et al. Gr00t n1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025.
- [8] G. R. Team, S. Abeyruwan, J. Ainslie, J.-B. Alayrac, M. G. Arenas, T. Armstrong, A. Balakrishna, R. Baruch, M. Bauza, M. Blokzijl, et al. Gemini robotics: Bringing ai into the physical world. *arXiv preprint arXiv:2503.20020*, 2025.
- [9] S. Liu, L. Wu, B. Li, H. Tan, H. Chen, Z. Wang, K. Xu, H. Su, and J. Zhu. Rdt-1b: a diffusion foundation model for bimanual manipulation. In *International Conference on Learning Representations*, volume 2025, pages 29982–30009, 2025.
- [10] P. Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.
- [11] L. Zha, A. J. Hancock, M. Zhang, T. Yin, Y. Huang, D. Shah, A. Z. Ren, and A. Majumdar. Lap: Language-action pre-training enables zero-shot cross-embodiment transfer. *arXiv preprint arXiv:2602.10556*, 2026.
- [12] A. Wagenmaker, M. Nakamoto, Y. Zhang, S. Park, W. Yagoub, A. Nagabandi, A. Gupta, and S. Levine. Steering your diffusion policy with latent space reinforcement learning. *arXiv preprint arXiv:2506.15799*, 2025.
- [13] A. Amin, R. Aniceto, A. Balakrishna, K. Black, K. Conley, G. Connors, J. Darpinian, K. Dhabalia, J. DiCarlo, D. Driess, et al. $\pi_{0.6}^*$: a vla that learns from experience. *arXiv preprint arXiv:2511.14759*, 2025.

- [14] Y. Li, X. Ma, J. Xu, Y. Cui, Z. Cui, Z. Han, L. Huang, T. Kong, Y. Liu, H. Niu, et al. Gr-rl: Going dexterous and precise for long-horizon robotic manipulation. *arXiv preprint arXiv:2512.01801*, 2025.
- [15] Z. Sun and S. Song. From prior to pro: Efficient skill mastery via distribution contractive rl finetuning. *arXiv preprint arXiv:2603.10263*, 2026.
- [16] M. Murray, D. Chen, S. Bagaria, D. Fortier, T. Hellebrekers, G. Mullins, H. Gajarla, O. Mees, M. Cakmak, and A. Kolobov. Flowdagger: Human-in-the-loop adaptation of generative robot policies in latent space. *arXiv preprint arXiv:2607.08877*, 2026.
- [17] H. Chen, A. Zhang, S. Schaefer, K. Chen, S. Chen, D. Cremers, O. Mees, and S. Leutenegger. Robot self-improvement via human-video dynamics models. *arXiv preprint arXiv:2606.21406*, 2026.
- [18] M. J. Kim, C. Finn, and P. Liang. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645*, 2025.
- [19] K. Black, A. Z. Ren, M. Equi, and S. Levine. Training-time action conditioning for efficient real-time chunking. *arXiv preprint arXiv:2512.05964*, 2025.
- [20] M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma, A. Zouitine, M. Aractingi, C. Pascal, M. Russi, A. Marafioti, et al. Smolvla: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*, 2025.
- [21] K. Black, M. Galliker, and S. Levine. Real-time execution of action chunking flow policies. *Advances in Neural Information Processing Systems*, 38:33383–33407, 2026.
- [22] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Choromanski, T. Ding, D. Driess, A. Dubey, C. Finn, P. Florence, C. Fu, M. G. Arenas, K. Gopalakrishnan, K. Han, K. Hausman, A. Herzog, J. Hsu, B. Ichter, A. Irpan, N. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, I. Leal, L. Lee, T.-W. E. Lee, S. Levine, Y. Lu, H. Michalewski, I. Mordatch, K. Pertsch, K. Rao, K. Reymann, M. Ryoo, G. Salazar, P. Sanketi, P. Sermanet, J. Singh, A. Singh, R. Soricut, H. Tran, V. Vanhoucke, Q. Vuong, A. Wahid, S. Welker, P. Wohlhart, J. Wu, F. Xia, T. Xiao, P. Xu, S. Xu, T. Yu, and B. Zitkovich. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In *arXiv preprint arXiv:2307.15818*, 2023.
- [23] J. Pai, L. Achenbach, V. Montesinos, B. Forrai, O. Mees, and E. Nava. mimic-video: Video-action models for generalizable robot control beyond vlas. In *Proceedings of Robotics: Science and Systems*, Sydney, Australia, 2026.
- [24] S. Ye, Y. Ge, K. Zheng, S. Gao, S. Yu, G. Kurian, S. Indupuru, Y. L. Tan, C. Zhu, J. Xiang, et al. World action models are zero-shot policies. *arXiv preprint arXiv:2602.15922*, 2026.
- [25] B. Hou, G. Li, J. Jia, T. An, X. Guo, S. Leng, H. Geng, Y. Ze, T. Harada, P. Torr, O. Mees, M. Pollefeys, Z. Liu, J. Wu, P. Abbeel, J. Malik, Y. Du, and J. Yang. World model for robot learning: A comprehensive survey. *The International Journal of Robotics Research*, 2026.
- [26] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn. Learning fine-grained bimanual manipulation with low-cost hardware. *arXiv preprint arXiv:2304.13705*, 2023.
- [27] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, page 02783649241273668, 2023.
- [28] S. Dasari, O. Mees, S. Zhao, M. K. Srirama, and S. Levine. The ingredients for robotic diffusion transformers. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, Atlanta, USA, 2025.

- [29] K. Pertsch, K. Stachowicz, B. Ichter, D. Driess, S. Nair, Q. Vuong, O. Mees, C. Finn, and S. Levine. Fast: Efficient action tokenization for vision-language-action models. In *Proceedings of Robotics: Science and Systems*, Los Angeles, USA, 2025.
- [30] Y. Liu, J. Hamid, A. Xie, Y. Lee, M. Du, and C. Finn. Bidirectional decoding: Improving action chunking via guided test-time sampling. In *International Conference on Learning Representations*, volume 2025, pages 4594–4627, 2025.
- [31] J. Tan, T. Zhang, E. Coumans, A. Iscen, Y. Bai, D. Hafner, S. Bohez, and V. Vanhoucke. Sim-to-real: Learning agile locomotion for quadruped robots. In *Robotics: Science and Systems*, 2018.
- [32] J. Hwangbo, J. Lee, A. Dosovitskiy, D. Bellicoso, V. Tsounis, V. Koltun, and M. Hutter. Learning agile and dynamic motor skills for legged robots. *Science Robotics*, 4(26):eaau5872, 2019.
- [33] E. Altman and P. Nain. Closed-loop control with delayed information. *IEEE Transactions on Automatic Control*, 48(4):553–558, 2003.
- [34] K. V. Katsikopoulos and S. E. Engelbrecht. Markov decision processes with delays and asynchronous cost collection. *IEEE Transactions on Automatic Control*, 48(4):568–574, 2003.
- [35] T. J. Walsh, A. Nouri, L. Li, and M. L. Littman. Planning and learning in environments with delayed feedback. *Autonomous Agents and Multi-Agent Systems*, 18(1):83–105, 2009.
- [36] E. Schuitema, L. Busoniu, R. Babuska, and P. Jonker. Control delay in reinforcement learning for real-time dynamic systems: A memoryless approach. In *2010 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 3226–3231. IEEE, 2010.
- [37] V. Firoiu, T. Ju, and J. B. Tenenbaum. At human speed: Deep reinforcement learning with action delay. *arXiv preprint arXiv:1810.07286*, 2018.
- [38] Y. Bouteiller, S. Ramstedt, G. Beltrame, C. J. Pal, and J. Binas. Reinforcement learning with random delays. In *International Conference on Learning Representations*, 2021.
- [39] T. Xiao, J. Ibarz, D. Kalashnikov, S. Levine, E. Jang, K. Hausman, and A. Herzog. Thinking while moving: Deep reinforcement learning with concurrent control. *arXiv preprint arXiv:2004.06089*, 2020.
- [40] L. Smith, I. Kostrikov, and S. Levine. A walk in the park: Learning to walk in 20 minutes with model-free reinforcement learning. *arXiv preprint arXiv:2208.07860*, 2022.
- [41] L. Smith, J. C. Kew, X. B. Peng, S. Ha, J. Tan, and S. Levine. Legged robots that keep on learning: Fine-tuning locomotion policies in the real world. In *2022 international conference on robotics and automation (ICRA)*, pages 1593–1599. IEEE, 2022.
- [42] H. Zhu, J. Yu, A. Gupta, D. Shah, K. Hartikainen, A. Singh, V. Kumar, and S. Levine. The ingredients of real-world robotic reinforcement learning. *arXiv preprint arXiv:2004.12570*, 2020.
- [43] J. Luo, Z. Hu, C. Xu, Y. L. Tan, J. Berg, A. Sharma, S. Schaal, C. Finn, A. Gupta, and S. Levine. Serl: A software suite for sample-efficient robotic reinforcement learning. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 16961–16969. IEEE, 2024.
- [44] R. Mendonca, E. Panov, B. Bucher, J. Wang, and D. Pathak. Continuously improving mobile manipulation with autonomous real-world rl. *arXiv preprint arXiv:2409.20568*, 2024.
- [45] E. Rosete-Beas, O. Mees, G. Kalweit, J. Boedecker, and W. Burgard. Latent plans for task agnostic offline reinforcement learning. In *Conference on Robot Learning (CoRL)*, 2022.

- [46] J. Luo, C. Xu, J. Wu, and S. Levine. Precise and dexterous robotic manipulation via human-in-the-loop reinforcement learning. *Science Robotics*, 10(105):eads5033, 2025.
- [47] M. Cutler, T. J. Walsh, and J. P. How. Reinforcement learning with multi-fidelity simulators. In *2014 IEEE International Conference on Robotics and Automation (ICRA)*, pages 3888–3895. IEEE, 2014.
- [48] A. Rajeswaran, S. Ghotra, B. Ravindran, and S. Levine. Epopt: Learning robust neural network policies using model ensembles. *arXiv preprint arXiv:1610.01283*, 2016.
- [49] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In *2017 IEEE/RSJ international conference on intelligent robots and systems (IROS)*, pages 23–30. IEEE, 2017.
- [50] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel. Sim-to-real transfer of robotic control with dynamics randomization. In *2018 IEEE international conference on robotics and automation (ICRA)*, pages 3803–3810. IEEE, 2018.
- [51] Y. Chebotar, A. Handa, V. Makoviychuk, M. Macklin, J. Issac, N. Ratliff, and D. Fox. Closing the sim-to-real loop: Adapting simulation randomization with real world experience. In *ICRA*, 2019.
- [52] J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter. Learning quadrupedal locomotion over challenging terrain. *Science robotics*, 5(47):eabc5986, 2020.
- [53] A. Kumar, Z. Fu, D. Pathak, and J. Malik. Rma: Rapid motor adaptation for legged robots. *arXiv preprint arXiv:2107.04034*, 2021.
- [54] A. Wagenmaker, K. Huang, L. Ke, K. Jamieson, and A. Gupta. Overcoming the sim-to-real gap: Leveraging simulation to learn to explore for real-world rl. *Advances in Neural Information Processing Systems*, 37:78715–78765, 2024.
- [55] Z. Zhang, K. Zheng, Z. Chen, J. Jang, Y. Li, S. Han, C. Wang, M. Ding, D. Fox, and H. Yao. Grape: Generalizing robot policy via preference alignment. *arXiv preprint arXiv:2411.19309*, 2024.
- [56] M. S. Mark, T. Gao, G. G. Sampaio, M. K. Srirama, A. Sharma, C. Finn, and A. Kumar. Policy agnostic rl: Offline rl and online rl fine-tuning of any class and backbone. *arXiv preprint arXiv:2412.06685*, 2024.
- [57] M. Nakamoto, O. Mees, A. Kumar, and S. Levine. Steering your generalists: Improving robotic foundation models via value guidance. *Conference on Robot Learning (CoRL)*, 2024.
- [58] Y. Chen, S. Tian, S. Liu, Y. Zhou, H. Li, and D. Zhao. Conrft: A reinforced fine-tuning method for vla models via consistency policy. *arXiv preprint arXiv:2502.05450*, 2025.
- [59] J. Hu, R. Hendrix, A. Farhadi, A. Kembhavi, R. Martín-Martín, P. Stone, K.-H. Zeng, and K. Ehsani. Flare: Achieving masterful and adaptive robot policies with large-scale reinforcement learning fine-tuning. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 3617–3624. IEEE, 2025.
- [60] L. Ankile, A. Simeonov, I. Shenfeld, M. Torne, and P. Agrawal. From imitation to refinement-residual rl for precise assembly. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 01–08. IEEE, 2025.
- [61] P. Dong, S. Mirchandani, D. Sadigh, and C. Finn. What matters for batch online reinforcement learning in robotics? *arXiv preprint arXiv:2505.08078*, 2025.
- [62] P. Dong, Q. Li, D. Sadigh, and C. Finn. Expo: Stable reinforcement learning with expressive policies. *arXiv preprint arXiv:2507.07986*, 2025.

- [63] W. Xiao, H. Lin, A. Peng, H. Xue, T. He, Y. Xie, F. Hu, J. Wu, Z. Luo, L. Fan, et al. Self-improving vision-language-action models with data generation via residual rl. *arXiv preprint arXiv:2511.00091*, 2025.
- [64] C. Xu, J. T. Springenberg, M. Equi, A. Amin, A. Esmail, S. Levine, and L. Ke. Rl token: Bootstrapping online rl with vision-language-action models. *arXiv preprint arXiv:2604.23073*, 2026.
- [65] Y. Guo, J. Zhang, X. Chen, X. Ji, Y.-J. Wang, Y. Hu, and J. Chen. Improving vision-language-action model with online reinforcement learning. *arXiv preprint arXiv:2501.16664*, 2025.
- [66] G. Lu, W. Guo, C. Zhang, Y. Zhou, H. Jiang, Z. Gao, Y. Tang, and Z. Wang. Vla-rl: Towards masterful and general robotic manipulation with scalable reinforcement learning. *arXiv preprint arXiv:2505.18719*, 2025.
- [67] V. Myers, B. C. Zheng, O. Mees, S. Levine, and K. Fang. Policy adaptation via language optimization: Decomposing tasks for few-shot imitation. In *Conference on Robot Learning*, 2024.
- [68] J. Liu, F. Gao, B. Wei, X. Chen, Q. Liao, Y. Wu, C. Yu, and Y. Wang. What can rl bring to vla generalization? an empirical study. *arXiv preprint arXiv:2505.19789*, 2025.
- [69] T. Johannink, S. Bahl, A. Nair, J. Luo, A. Kumar, M. Loskyll, J. A. Ojea, E. Solowjow, and S. Levine. Residual reinforcement learning for robot control, 2018. URL <https://arxiv.org/abs/1812.03201>.
- [70] X. Yuan, T. Mu, S. Tao, Y. Fang, M. Zhang, and H. Su. Policy decorator: Model-agnostic online refinement for large policy model. *arXiv preprint arXiv:2412.13630*, 2024.
- [71] L. Ankile, Z. Jiang, R. Duan, G. Shi, P. Abbeel, and A. Nagabandi. Residual off-policy rl for finetuning behavior cloning policies. *arXiv preprint arXiv:2509.19301*, 2025.
- [72] Q. Li, S. Park, and S. Levine. Decoupled q-chunking. *arXiv preprint arXiv:2512.10926*, 2025.
- [73] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn. Learning fine-grained bimanual manipulation with low-cost hardware, 2023. URL <https://arxiv.org/abs/2304.13705>.
- [74] M. Matthews, M. Beukman, C. Lu, and J. Foerster. Kinetix: Investigating the training of general agents through open-ended physics-based control tasks, 2025. URL <https://arxiv.org/abs/2410.23208>.
- [75] Physical Intelligence. openpi. <https://github.com/Physical-Intelligence/openpi>, 2025. GitHub repository, accessed June 2026.

Contributions

- Brian Zhu: Methodology (contribution 2: adding intermediate state), DSRL + RTC repro, Aloha simulation, software architecture & implementation, writing
- Momen Khalil: Methodology (contribution 1: adding intermediate actions), DSRL + RTC repro, Kinetix simulation, software architecture & implementation, writing
- E Harrison: Simulated experiments, writing
- Emanuele Poggi: Simulation and real-world experiments, ran learning dynamics analysis, writing
- Philipp Schmitt: original idea, software architecture, advice
- Bernd Kast: Software architecture & implementation, advice
- Philine Meister: Real-world experiment lead, writing
- Pranav Atreya: Experimental support, writing
- Qiyang Li: Theoretical results
- Finn Ferchau: Real-world experiment execution
- Cesar Colmenero: Real-world experiment execution
- Yash Shahapurkar: Early real-world experiment testing
- Gokul Narayanan: Early real-world experiment testing
- Melih Erdogan: Early real-world experiment testing
- Kai Wurm: Advice, resources, funding
- Georg von Wichert: Advice, resources, funding
- Oier Mees: Advising, writing
- Eugen Solowjow: Advice, resources, funding
- Andrew Wagenmaker: Advising, method ideation, writing
- Sergey Levine: Advising, writing

A Experimental Details

A.1 Implementation

Algorithm. For all experiments, we run ARLI and DSRL using DSRL-SAC with repeated latent actions as described in the π_0 experiments from [12]. In particular, we train a single step actor $\pi_{\text{rl}}^{\text{single}}$ and critic $Q^{\text{single}}(s, w_{\text{single}})$ where $\mathcal{W}_{\text{single}} = \mathbb{R}^d$; during inference we sample $w_{\text{single}} \sim \pi_{\text{rl}}^{\text{single}}(s)$ and repeat it across the action chunk axis to query the flow policy, i.e. $w = \{(w_1, \dots, w_k) | w_i = w_{\text{single}}\}$ where k is the chunk length of the flow policy. Similar to [12], we handle RL with action chunking by treating the sampling and execution of an action chunk as a single step in the latent MDP, ignoring observations from within the chunk. Note that the number of environment steps stated in all results corresponds to the total number of steps from the original environment rather than the latent MDP.

For residual RL experiments, we use the implementation described in [71]. Specifically, we train residual policy π_{rl} to output a residual correction a_t^Δ , which is used to edit the base policy’s action and execute $\tilde{a}_t = a_t + a_t^\Delta$. The resulting action $\tilde{a}_t$ is also used to train the critic function $Q(s_t, \tilde{a}_t)$. The residual RL policy can be run independently of the base policy’s inference process, and due to its lightweight size, can essentially be run without inference delay. So, we provide π_{rl} with the most up-to-date RL state, $s_t^{\text{rl}} \leftarrow s_t$. Unlike DSRL which necessarily stays within the support of the base policy, residual edits that have a similar magnitude to the base policy actions can lead to the policy relearning the task from scratch, rather than finetuning the base policy. To ensure that this does not occur, we limit the search of the optimal action magnitude for residual edits (b_w) within $[0, 0.1]$.

Observation Space and Architecture. For Kinetix tasks we use the "symbolic" observation space, which according to [74] is a flat vector that concatenates the physical properties all the entities in the scene (e.g., position, rotation, velocity). The actor and critics are thus defined as an MLP that processes this observation vector. For AlohaTransferCube and real-world tasks we use the raw images and joint state as the observation space. For both actor and critics, the raw images are processed by a vision encoder consisting of 4 CNN layers before being flattened and concatenated with the joint state. The final state vector is then processed by an MLP. Table 1 summarizes the observation space and architecture choices for each task. Note that for real-world tasks, we use 3 camera streams (1 base camera along with 1 wrist camera for each arm); the images are concatenated along the channel dimension before being processed by the vision encoder.

Table 1: Observation space and architecture used for each environment

Hyperparameter	Kinetix	AlohaTransferCube	Real-World
Observation space	Symbolic	Pixels & State	Pixels & State
Image observation size	N/A	64×64	64×64
Actor and critic architecture	MLP	CNN+MLP	CNN+MLP
Number of actor and critic MLP layers	3	3	3
MLP hidden size	256	128	128
CNN features	N/A	(32, 32, 32, 32)	(32, 32, 32, 32)
Number of images	N/A	1 (cam_high)	3 (base & wrists)

Intermediate Information Processing. To handle the intermediate state introduced by ARLI, we concatenate the intermediate state with the original state. For Kinetix, we concatenate the observation vectors. For AlohaTransferCube and real-world tasks, we concatenate images along the channel dimension and separately concatenate the joint states. To handle the intermediate actions introduced by ARLI, we flatten the actions and concatenate them to the full state vector before it is processed by the MLP layers.

Code. We use the dsrl_pi0 codebase from [12] to implement the DSRL-SAC training loop. We modify dsrl_pi0 and openpi [75] to integrate RTC based on the codebase from [21]. We also integrate the Kinetix tasks from [21] with DSRL. For AlohaTransferCube, we do not make additional modifications.

Base Policy. For Kinetix, we use the BC flow policy checkpoints from [21] for each task. In particular, we use the checkpoint for the 5th epoch as the base policy initialization. For AlohaTransferCube, we follow [12] and use the π_0 checkpoint from s3://openpi-assets/checkpoints/pi0_aloha_sim. For real-world experiments, we finetune $\pi_{0.5}$ [10] with LoRA on each task using a cosine decay schedule. Training data was recorded for the specific tasks at 60Hz using a Meta Quest to remote control the robots. Note that the training data for the Shoe-in-Bag task does not only focus on packing the shoe, but also contains demonstrations for packing other clothing items as t-shirts into the bag. Dataset details and training hyperparameters for finetuning $\pi_{0.5}$ are listed in Table 2.

A.2 Hyperparameters

DSRL-specific hyperparameters are listed in Table 3. Simulation evaluation metrics are averaged over 50 rollouts. For all experiments with RTC, we use the hyperparameters listed in Table 4. Values of inference delay we ablate over are shown as a list. Residual RL hyperparameters are listed in Table 5.

We initialize all experiments with offline data collection (that is, rolling out the base policy some number of episodes) and optionally offline training (that is, training the actor and critic with SAC on the offline data for some number of steps). Hyperparameters are listed in Table 6. Note that number of offline transitions refer to transitions from the latent MDP rather than the original environment. For AlohaTransferCube, we found that scaling down the variance of the noise distribution boosted

Table 2: Real-world experiments base policy training data details

	Assembly	Shoe-in-Bag	Bag-Placement
Number of episodes	60	123	51
Total dataset steps	84821	474301	50041
Batch size	32	32	32
Approx. number of epochs	3	2	4
Warm-up steps	265	1000	1000
Decay steps	8000	30000	30000
Training steps	8000	30000	6000
Peak-LR	2.5e-5	2.5e-5	2.5e-5
Decay-LR	2.5e-6	2.5e-6	2.5e-6

the number of successful trajectories in the replay buffer. Similarly, for real-world experiments we found that clipping the sampled noise w_{single} reduces unsafe motions from the robot.

Table 3: Hyperparameters used for DSRL

Hyperparameter	Kinetix	AlohaTransferCube	Real-World
Batch size	256	256	256
Actor learning rate	0.0001	0.0001	0.0001
Critic learning rate	0.0003	0.0003	0.0003
Temperature learning rate	0.0003	0.0003	0.0003
Activation	ReLU	ReLU	ReLU
Discount factor	0.999	0.999	0.999
Target entropy	$-d/2$	$-d/2$	$-d/2$
Target update rate (τ)	0.005	0.005	0.005
Gradient steps per update	20	25	20
Action magnitude (b_w)	1.0	2.0	0.5
Action chunk size	8	50	50
Number of actions to execute	4	25	20
Number of critics	10	10	10
Clipped Q-learning	False	False	False

Table 4: Hyperparameters used for RTC

Hyperparameter	Kinetix	AlohaTransferCube	Real-World
Denoising steps	5	10	10
Inference Delay (t_{delay})	[1, 2, 3, 4]	[10, 20]	10
Soft Masking	True	True	True
Max. Guidance Weight	5	10	10

Table 5: Hyperparameters used for Residual RL

Hyperparameter	Kinetix	AlohaTransferCube
Batch size	256	256
Actor learning rate	0.000001	0.000001
Critic learning rate	0.0001	0.0001
Activation	ReLU	ReLU
Discount factor	0.99	0.99
Target update rate (τ)	0.005	0.005
Gradient steps per update	4	4
Action magnitude (b_w)	0.1	0.1
Action chunk size	8	50
Number of actions to execute	4	25
Number of critics	10	10
Clipped Q-learning	False	False

Table 6: Hyperparameters used for initial offline data collection

Hyperparameter	Kinetix	AlohaTransferCube	Real-World
Number of offline transitions	500	1000	1200
Offline training steps	0	0	24000
Offline noise distribution	$\mathcal{N}(0, 1)$	$\mathcal{N}(0, 0.3)$	$\mathcal{N}(0, 1)$
Clip noise	False	False	True (0.5)

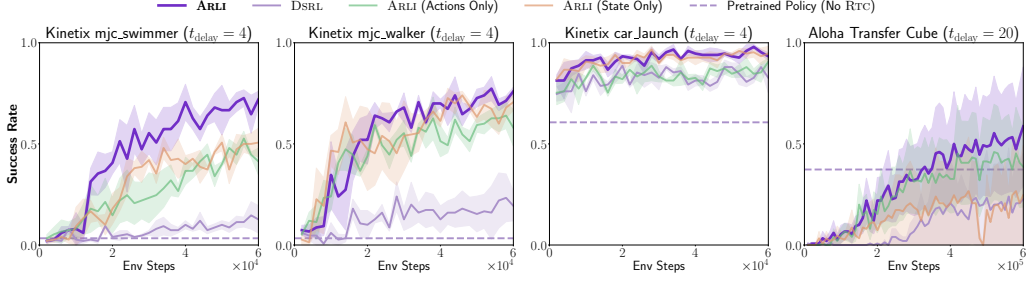


Figure 9: Variants of ARLI (no RTC) with one of intermediate state or actions on mjc_swimmer, mjc_walker, car_launch and AlohaTransferCube.

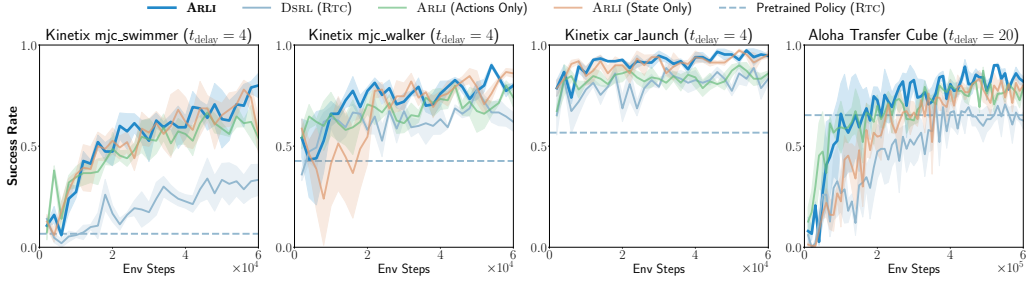


Figure 10: Variants of ARLI (with RTC) with one of intermediate state or actions on mjc_swimmer, mjc_walker, car_launch and AlohaTransferCube.

B Additional Ablations

We run additional ablations from Section 5.3 for the following questions.

How critical is it that we condition π_{r1} on the intermediate state and actions? We run the same ablation on all tasks from the main results from Section 5.1 (mjc_swimmer, mjc_walker, car_launch, AlohaTransferCube) and also ablate between running ARLI with and without RTC. Figure 9 shows ablation results without RTC and Figure 10 shows ablation results with RTC. Adding RTC reduces the amount of additional information that needs to be added to final state representation, but across all tasks and environments we see that including all intermediate information consistently produces the best results.

How sensitive is ARLI to t_{delay}^{r1} ? We run the same set of ablations over t_{delay}^{r1} on mjc_swimmer with and without RTC, Figure 11 shows the results of the experiments. Here we see that removing RTC increases the requirement for how up-to-date the intermediate state must be. Between RTC and

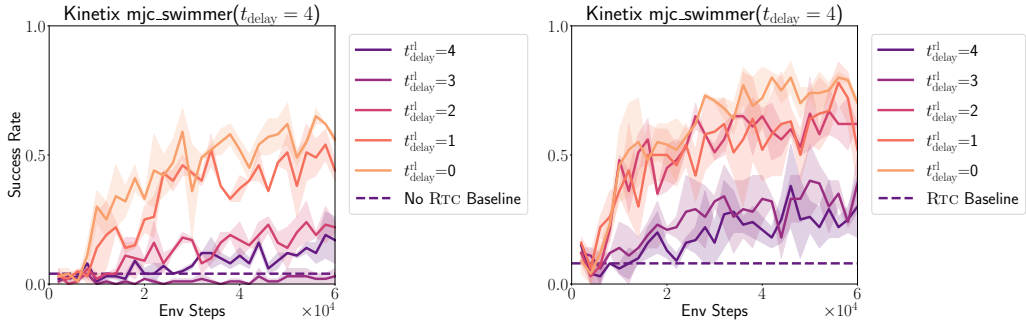


Figure 11: ARLI performance at varying RL inference delays on mjc_swimmer. *Left*: without RTC. *Right*: with RTC.

no RTC we see that significant improvements in policy performance only appear when $t_{\text{delay}}^{\text{rl}} < 3$ and $t_{\text{delay}}^{\text{rl}} < 2$, respectively.

C Learning Dynamics of ARLI Noise Steering

To better understand the learning behaviour of ARLI during training, we analyze the learned ARLI policy on the real-world Shoe-in-Bag task over four held-out episodes not seen from the model during training.

In this diagnostic, we focus on the ARLI variant conditioned on previous actions only, since the considered held-out trajectories saved the previous-actions information, but not the mid-inference observation; we therefore use the previous-action model so that every checkpoint is evaluated on exactly the same modalities used for training. For Shoe-in-Bag, the learning curves of full ARLI and the previous-action-only variant are similar, making this a useful diagnostic for the learned steering behavior. We replay two successful and two failed held-out Shoe-in-Bag episodes through every trained ARLI checkpoint. For each replayed inference step, we record the mean and variance of the noise distribution predicted by ARLI before it is denoised by the action expert. The first checkpoint considered is after the 24000 offline training steps, corresponding to the first model trained after collecting the initial warm-up episodes in the replay buffer.

The middle column of Figure 12 visualizes how the predicted mean and variance of the policy’s learned noise distribution evolve across training checkpoints for the replayed trajectories. The main cross-episode trend is that failed episodes exhibit a directional shift in the predicted noise distribution that is not present in the same way for successful episodes. On the held-out failures, later ARLI checkpoints produce a more displaced and somewhat narrower steering-noise distribution: the predicted noise mean moves farther from zero while the predicted variance decreases. This pattern is consistent with later ARLI checkpoints becoming more confident and more biased in particular regions of the failed trajectories.

This shift is localized in time rather than uniformly spread over the whole trajectory. In the heatmaps, the increased mean magnitude and reduced variance are most visible around the phase that requires the largest corrective steering. The first successful episode is smooth and well aligned, so there is little need for ARLI to apply a large steering correction; accordingly, it shows the weakest localized signal among the four examples. The second successful episode is still successful but less smooth, which is consistent with the stronger mid-trajectory signal visible in the plots. The first failed episode is close to success: the shoe is only slightly misaligned with the bag opening, but the remaining correction is difficult and ultimately leads to failure. This helps explain why the signal for this episode becomes more pronounced later in training, when the learned steering distribution has become more specialized. The second failed episode represents the typical failure mode of this task, often observed in early failed episodes, where the shoe is substantially misaligned with the bag entrance. In this case, the learned signal is visible earlier in the checkpoint sequence, suggesting that this failure mode is easier for ARLI to identify.

The latent-dimension view in the right column of Figure 12 provides a coarse diagnostic of where the checkpoint-dependent changes occur in the learned noise space. To quantify this, we compute the symmetric KL contribution of each latent dimension between the first and final checkpoints, averaged over inference steps. Overall, the drift remains broadly distributed across the latent dimensions, with effective dimension counts between 22.7 and 26.7 out of 32, suggesting that the learned change is not concentrated in only one or two specific coordinates. Failed 2 shows the clearest, though still moderate, concentration: its top three latent dimensions explain 25.3% of the total early-to-late KL contribution, compared with roughly 19–20% for the other episodes. We can therefore interpret this mainly as evidence that the learned change is distributed across the latent noise space, with only limited dimension-level localization.

Overall, this analysis suggests that ARLI does not merely rescale the pretrained policy’s noise input globally, rather it reshapes the noise distribution in a trajectory- and phase-dependent manner. The considered successful episodes show weak localized distributional-drift, while the failed episodes are distinguished by the combination of three effects: the predicted noise mean moves farther from zero, the predicted variance decreases more strongly, and the largest distributional changes concentrate around replay steps corresponding to the middle alignment/insertion portion of the task.

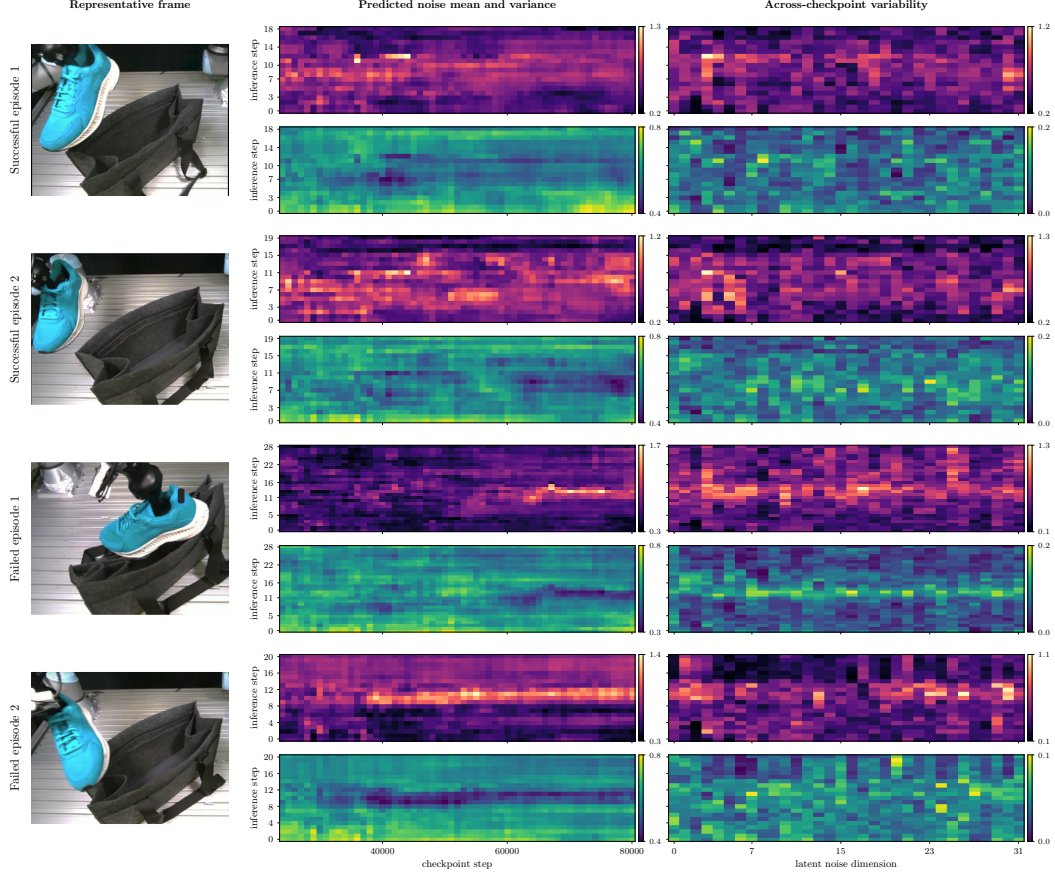


Figure 12: Checkpoint-wise evolution of the ARLI (actions only) noise distribution on four held-out Shoe-in-Bag episodes. *Left*: a representative frame from the middle of each trajectory. *Middle*: predicted noise mean (upper plot) and variance (lower plot) by inference step and checkpoint, averaged over latent noise dimensions. *Right*: across-checkpoint standard deviation of the predicted noise mean (upper plot) and variance (lower plot) by inference step and latent noise dimension.

This is consistent with the qualitative observation that failures in Shoe-in-Bag often occur around alignment and insertion rather than at the very beginning of the episode. The findings also align with the role of ARLI’s intermediate information: because it observes the actions that will be executed during inference and, in the full method, can also use a more recent intermediate state, it can learn steering corrections that are targeted to the effective state at which the next action chunk will be used.

We emphasize that this analysis is intentionally conservative: since it uses only four held-out trajectories replayed through multiple checkpoints of a single task, we interpret it as descriptive evidence about how the learned noise distribution evolves on these episodes, not as a population-level success/failure classifier.

D Theoretical Results

Our main theoretical result (Proposition 1) is built on top of the framework in Li et al. [72].

We first start by describing all the existing assumptions (Assumption 1 from Li et al. [72]) and a crucial new assumption (definition 2) that quantifies the sub-optimality caused by inference delays.

Assumption 1 (Data Obeys the Transition Dynamics) $\mathcal{D} \in \Delta_{\mathcal{T}}$ is a trajectory distribution generated by rolling out a behavior policy from a distribution of $s_t \sim \mu$. The behavior policy can be non-Markovian (i.e., $\pi_{\beta}(a_{t+h} \mid s_{t:t+h+1}, a_{t:t+h})$). Each subsequent state is generated according to the dynamics of the MDP $\mathcal{M}$: $s_{t+h+1} \sim P(\cdot \mid s_{t+h}, a_{t+h}), \forall h \in \{0, 1, \dots, k-1\}$. The resulting trajectory is $(s_t, s_{t+1}, \dots, s_{t+k}, a_t, a_{t+1}, \dots, a_{t+k}) \in \mathcal{T} = \mathcal{S}^k \times \mathcal{A}^k$.

Definition 1 (Strong Open-Loop Consistency) $\mathcal{D}$ is strongly open-loop consistent if for every $a_{t:t+k} \in \text{supp}(P_{\mathcal{D}}(a_{t:t+k} \mid s_t))$,

$$P(s_{t+k'} \mid s_t, a_{t:t+k'}) = P_{\mathcal{D}}(s_{t+k'} \mid s_t, a_{t:t+k}), \quad \forall k' \in \{1, 2, \dots, k\}, \quad (2)$$

where we use $P(s_{t+k'} \mid s_t, a_{t:t+k'})$ to denote the distribution of the future state $s_{t+k'}$ after carrying out the action sequence $a_{t:t+k'}$ in the environment open-loop from s_t .

Notations. Following the notations introduced by Li et al. [72], we learn an action chunking Q-function, $\hat{Q}_{\text{ac}}^+(s_t, a_{t:t+k})$. In our setting, there is a d -time-step delay on the observation s_t , so we may only react to the observation information that is d -step delayed (i.e., s_{t-d}) when carrying out a_t . Our policy approximates the best action chunk ($a_{t-d:t-d+k}$) that retains the same ‘‘action prefix’’ ($a_{t-d}^{\bullet}, \dots, a_{t-1}^{\bullet}$) while maximizing $\hat{Q}_{\text{ac}}^+(s_{t-d}, a_{t-d:t-d+k})$. We formalize this as follows:

$$a_{t:t+k-d}^{\circ} = \arg \max_{\hat{a}_{t:t+k-d}} \hat{Q}_{\text{ac}}^+(s_{t-d}, \hat{a}_{t-d:t+k-d}), \quad (3)$$

where

$$\hat{a}_{t-d:t+k-d} := \underbrace{a_{t-d}^{\bullet}, a_{t-d+1}^{\bullet}, \dots, a_{t-1}^{\bullet}}_{\text{not used due to time delay}}, \underbrace{\hat{a}_t, \hat{a}_{t+1}, \dots, \hat{a}_{t+k-d-1}}_{\text{open-loop execution}} \quad (4)$$

We denote the policy that produces a° the d -delayed action chunking policy:

$$\pi_d(s_{t-d}) := a_{t:t+k-d}^{\circ} \quad (5)$$

Lemma 1 (Open-loop consistency of d -delayed policy) If $\mathcal{D}$ satisfies Assumption 1 and it is also collected by rolling out an d -delayed action chunking policy for any $d \geq 0$, that is

$$P_{\mathcal{D}}(a_{t+d:t+k} \mid s_{t:t+k}) = P_{\mathcal{D}}(a_{t+d:t+k} \mid s_t) = \delta_{\pi(s_t)}, \quad (6)$$

then $\mathcal{D}$ is open-loop consistent.

Proof. $P_{\mathcal{D}}(a_{t+d:t+k} \mid s_{t:t+k}) = P_{\mathcal{D}}(a_{t+d:t+k} \mid s_t)$ implies that the action chunk $a_{t+d:t+k}$ is independent from the intermediate actions $s_{t+1:t+k}$. In addition, we also know that $a_{t:t+d}$ have already been determined before s_t and cannot be changed after observing $s_{t:t+k}$ either. Therefore, we have

$$P_{\mathcal{D}}(a_{t:t+k} \mid s_{t:t+k}) = P_{\mathcal{D}}(a_{t:t+k} \mid s_t). \quad (7)$$

This allows us to establish

$$P_{\mathcal{D}}(s_{t+k'} \mid s_t, a_{t:t+k}) = P(s_{t+k'} \mid s_t, a_{t:t+k}), \quad \forall k' \in \{1, 2, \dots, k\} \quad (8)$$

as desired. $\square$

Lemma 2 (Action chunking Q-learning is optimal under d -delayed action chunking policy data collection) Let $\mathcal{D}$ be some data distribution collected by a mixture of d -delayed policies, and $a_{t:t+k}^* \in \text{supp}(P_{\mathcal{D}}(a_{t:t+k} | s_t))$ for $a_{t:t+k}^* = \arg \max_{a_{t:t+k}} Q_{\text{ac}}^*(s_t, a_{t:t+k})$. Then

$$\hat{Q}_{\text{ac}}^+(s_t, a_{t:t+k}) = Q_{\text{ac}}^*(s_t, a_{t:t+k}), \quad \forall s_t, a_{t:t+k} \in \text{supp}(P_{\mathcal{D}}), \quad (9)$$

where $\hat{Q}_{\text{ac}}^+$ is the learned value of the non-delayed action chunking policy: $\pi_{\text{ac}}^+ : s_t \mapsto \arg \max_{a_{t:t+k}} \hat{Q}_{\text{ac}}^+(s_t, a_{t:t+k})$, and Q_{ac}^* is the optimal value achievable by a non-delayed action chunking policy.

Proof. We first observe that any mixture of open-loop consistent data distribution is also open-loop consistent. It is not hard to conclude that $\mathcal{D}$ is open-loop consistent due to lemma 1. Now, we can reuse Theorem 1 from Li et al. [72] to conclude that

$$\hat{Q}_{\text{ac}}^+(s_t, a_{t:t+k}) = Q_{\text{ac}}^+(s_t, a_{t:t+k}), \quad \forall s_t, a_{t:t+k} \in \text{supp}(P_{\mathcal{D}}), \quad (10)$$

where Q_{ac}^+ is the true value of the non-delayed action chunking policy π_{ac}^+ .

Now, since we have assumed that the optimal action chunking policy data is in the support, the value maximizing action chunk must be the optimal action chunk. We can now conclude that

$$\hat{Q}_{\text{ac}}^+(s_t, a_{t:t+k}) = Q_{\text{ac}}^*(s_t, a_{t:t+k}), \quad \forall s_t, a_{t:t+k} \in \text{supp}(P_{\mathcal{D}}) \quad (11)$$

as desired. $\square$

The implication of this result is that we can use the data collected by any d -delayed action chunking policy to learn optimal action chunking Q-function as long as the data covers some optimal action chunking policy's behavior. However, even with the optimal action chunking Q-function, such optimal action chunking policy is not achievable because we have a delay of d time steps in our setting.

Definition 2 (Delayed Oracle Optimality Gap) An MDP $\mathcal{M}$ exhibits ω_d -delayed optimality gap if for any $s_t, a_{t:t+k}$,

$$|Q_{\text{ac}}^*(s_t, a_{t:t+k}) - \mathbb{E}_{P(\cdot | s_t, a_{t:t+k})}[R_{t:t+k-d} + \gamma^{h-d} V_{\text{ac}}^*(s_{t+k-d}, a_{t+k-d:t+k})]| \leq \omega_d, \quad (12)$$

where $V_{\text{ac}}^*(s_t, a_{t:t+d}) = \max_{a_{t+d:t+k}} Q_{\text{ac}}^*(s_t, a_{t:t+k})$.

Intuitively, ω_d upper-bounds the amount of sub-optimality resulted from a delayed decision. The left-hand-side (LHS) of the inequality quantifies the maximum value achievable when the decision is not delayed and at a regular interval of k (i.e., make decision at $s_t, s_{t+k}, \dots$). The right-hand-side (RHS) of the inequality quantifies the maximum value achievable when the decision is delayed (i.e., $a_{t+k:t+2k-d}$ is based on the delayed s_t) and because of the delay, the prefix of the action chunk is pre-determined (i.e., $a_{t+k-d:t+k}$).

Theorem 1 (Delayed Action Chunking Policy is Near-optimal) Let $\mathcal{D}$ be some data distribution collected by a mixture of d -delayed policies, and the following two support assumptions hold:

1. Optimal action chunks are in-support:

$$a_{t:t+k}^* \in \text{supp}(P_{\mathcal{D}}(\cdot | s_t)), \quad (13)$$

where $a_{t:t+k}^* = \pi_{\text{ac}}^*(s_t)$.

2. Optimal action chunk *completions* for in-support delayed action prefix are also in-support:

$$a_{t+d:t+k}^* \in \text{supp}(P_{\mathcal{D}}(\cdot | s_t, a_{t:t+d})), \quad \forall a_{t:t+d} \in \text{supp}(P_{\mathcal{D}}(\cdot | s_t)), \quad (14)$$

where $a_{t+d:t+k}^* = \arg \max_{a_{t+d:t+k}} Q_{ac}^*(s_t, a_{t:t+k})$.

If additionally, the MDP exhibits ω_d -delayed optimality gap, then the value of the d -delayed action chunking policy π_d^+, V_d^+ satisfies

$$|V_d^+(s_t, a_{t:t+d}) - V_{ac}^*(s_t, a_{t:t+d})| \leq \frac{\omega_d}{1 - \gamma^{k-d}}, \quad \forall s_t, a_{t:t+d} \in \text{supp}(P_{\mathcal{D}}), \quad (15)$$

where $V_{ac}^*(s_t, a_{t:t+d}) := \max_{a_{t+d:t+k}} Q_{ac}^*(s_t, a_{t:t+k})$ and $V_d^+(s_t, a_{t:t+d}) := Q_d^+(s_t, \tilde{a}_{t:t+k})$ with $\tilde{a}_{t:t+k} = [a_{t:t+d}, a_{t+d:t+k}^*]$, $a_{t+d:t+k}^* = \arg \max_{a_{t+d:t+k}} Q_{ac}^+(s_t, a_{t:t+k})$, and Q_d^+ is the fixed point of the following bellman equation:

$$Q_d^+(s_t, a_{t:t+k}) = \mathbb{E}_{P(\cdot|s_t, a_{t:t+k})}[R_{t:t+k-d} + \gamma^{k-d} Q_d^+(s_{t+k-d}, \tilde{a}_{t+k-d:t+2k-d})], \quad (16)$$

where again $\tilde{a}_{t+k-d:t+2k-d} := [a_{t+k-d:t+k}, a_{t+k:t+2k-d}^*]$ with $a_{t+k:t+2k-d}^* = \arg \max_{a_{t+k:t+2k-d}} Q_{ac}^+(s_{t+k-d}, a_{t+k-d:t+2k-d})$

Proof. We first note that due to the first support assumption (eq. (13)), we can trigger lemma 2 to conclude that $\hat{Q}_{ac}^+(s_t, a_{t:t+k}) = Q_{ac}^*(s_t, a_{t:t+k})$ for in-support $s_t, a_{t:t+k}$. On top of that, due to the second support assumption (eq. (14)), we know that the optimal action chunk completion would also be in-support, which means that

$$\arg \max_{a_{t+d:t+k}} \hat{Q}_{ac}^+(s_t, a_{t:t+k}) = \arg \max_{a_{t+d:t+k}} Q_{ac}^*(s_t, a_{t:t+k}), \quad \forall s_t, a_{t:t+d} \in \text{supp}(P_{\mathcal{D}}), \quad (17)$$

so we can use $a_{t+d:t+k}^*$ to denote the optimal action chunk completion for both value function.

We first write out

$$Q_d^+(s_t, a_{t:t+k}) = \mathbb{E}_{P(\cdot|s_t, a_{t:t+k})}[R_{t:t+k-d} + \gamma^{k-d} Q_d^+(s_{t+k-d}, [a_{t+k-d:t+k}, a_{t+k:t+2k-d}^*])], \quad (18)$$

and compare with

$$X = \mathbb{E}_{P(\cdot|s_t, a_{t:t+k})}[R_{t:t+k-d} + \gamma^{k-d} Q_{ac}^*(s_{t+k-d}, [a_{t+k-d:t+k}, a_{t+k:t+2k-d}^*])]. \quad (19)$$

From the ω_d -delayed optimality assumption, we know that

$$|X - Q_{ac}^*(s_t, a_{t:t+k})| \leq \omega_d. \quad (20)$$

Set $\tilde{a}_{t+k-d:t+2k-d} = [a_{t+k-d:t+k}, a_{t+k:t+2k-d}^*]$. We can now bound the difference between Q_d^+ and $Q_{ac}^*(s_t, a_{t:t+k})$.

$$|Q_d^+(s_t, a_{t:t+k}) - Q_{ac}^*(s_t, a_{t:t+k})| \quad (21)$$

$$\leq \mathbb{E}_{P(\cdot|s_t, a_{t:t+k})}[\omega_d + \gamma^{k-d} |Q_d^+(s_{t+k-d}, \tilde{a}_{t+k-d:t+2k-d}) - Q_{ac}^*(s_{t+k-d}, \tilde{a}_{t+k-d:t+2k-d})|] \quad (22)$$

$$\leq \frac{\omega_d}{1 - \gamma^{k-d}}. \quad (23)$$

This means that

$$|V_d^+(s_t, a_{t:t+d}) - V_{ac}^*(s_t, a_{t:t+d})| \leq \frac{\omega_d}{1 - \gamma^{k-d}}, \quad \forall s_t, a_{t:t+d} \in \text{supp}(P_{\mathcal{D}}). \quad (24)$$

□